Python 基础与模型泛化性实验

姓名　顾凯琪

学号　226001624

学院 信息学院

专业班级 计算机 2 班

一：实验目的

1. 学习掌握相关理论知识，例如模型的泛化性定义、过拟合、欠拟合等，并通过编程，从实践的角度提高对模型泛化性的理解；

2. 掌握 anaconda 环境、conda 工具的安装与使用，并学习使用 sklearn、matplotlib 等常见 Python 库函数；

3. 掌握机器学习模型训练的常规流程，包括数据集处理与划分、模型的训练、模型的验证等步骤，并理解学习各步骤的代码实现方法；

4. 总结归纳实验流程，理解过拟合、欠拟合等现象的发生原理、实验状况以及解决方案。

二：实验环境

操作系统：

Windows 11 Version 22H2

集成开发环境：

1. Anaconda3 with Jupyter Notebook
2. PyCharm

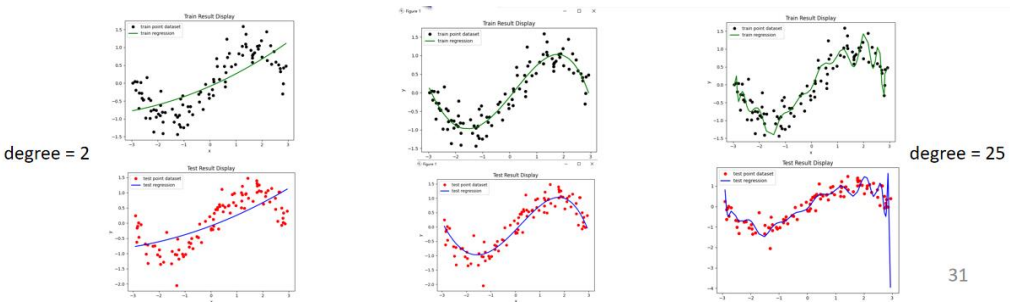
相关 Python 库：

1. numpy
2. matplotlib（3.8.3）
3. sklearn（scikit-learn1.4.1）
4. pandas（2.2.1）

三：实验内容

1. 用 sklearn 框架实现回归任务中的拟合过程，复现模型欠拟合、过拟合、正好拟合的状态。

2. 提交实验报告（Python 实现），包含相关语句注释，以及执行结果截图，将类似如下的结果图片贴到实验报告中。



四：实验步骤

1.环境搭建

Win11 系统中安装 python3.9，然后使用 pip 安装所需要的各个库。

命令如下：

pip install 库名

```
PS C:\Users\asus\Desktop\x\试卷卷卷子\机器学习\实验\实验课1 Python基础_泛化性> pip install scipy
Requirement already satisfied: scipy in d:\anaconda\lib\site-packages (1.2.2)
Requirement already satisfied: numpy>=1.8.2 in d:\anaconda\lib\site-packages (from scipy) (1.18.5)
Requirement already satisfied: pytz in d:\anaconda\lib\site-packages (from matplotlib) (2018.5)
Requirement already satisfied: six>=1.10 in d:\anaconda\lib\site-packages (from matplotlib) (1.16.0)
Requirement already satisfied: kiwisolver>=1.0.1 in d:\anaconda\lib\site-packages (from matplotlib) (1.0.1)
Requirement already satisfied: setuptools in d:\anaconda\lib\site-packages (from kiwisolver>=1.0.1->matplotlib) (68.0.0)
PS C:\Users\asus\Desktop\x\试卷卷卷子\机器学习\实验\实验课1 Python基础_泛化性> pip install numpy
Requirement already satisfied: numpy in d:\anaconda\lib\site-packages (1.18.5)
PS C:\Users\asus\Desktop\x\试卷卷卷子\机器学习\实验\实验课1 Python基础_泛化性>
```

2. 生成随特定函数分布的几百个随机数据点，例如 y=sin(x)；

3. 将其分为训练集和测试集，并可视化数据点；

4. 利用 sklearn 框架拟合；

提示：sklearn.preprocessing.PolynomialFeatures()

官方文档：https://scikit-learn.org/stable/

5. 改变传入 PolynomialFeatures()中的系数来调整曲线拟合程度；

6. 可视化拟合结果；（三幅图：欠拟合、过拟合、正常拟合）

五：参考代码

"""

--------生成数据--------

"""

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# 设置随机种子，以便结果可重复
random_state = np.random.RandomState(42)

# 生成训练数据和测试数据
x_train = random_state.uniform(-3, 3, size=100)  # 在区间[-3, 3]之间随机生成100个数作为训练数据
y_train = np.sin(x_train) + random_state.normal(size=len(x_train)) / 3  # 训练数据的标签，加上噪声
x_test = random_state.uniform(-3, 3, size=100)  # 测试数据
y_test = np.sin(x_test) + random_state.normal(size=len(x_test)) / 3  # 测试数据的标签，加上噪声

# 绘制数据图
fig, ax = plt.subplots(1)  # 创建单个子图
ax.scatter(x_train, y_train, color='black', label='Training Set')  # 绘制训练集数据点
ax.scatter(x_test, y_test, color='red', label='Test Set')  # 绘制测试集数据点
ax.legend(loc="best")  # 设置图例位置

# 设置坐标轴标签和标题
ax.set_ylabel("Output Value")
ax.set_xlabel("Input Value")
ax.set_title("Scatter Plot of Data Sets")

# 自动调整子图参数，以防止标签被遮挡
plt.tight_layout()
plt.show()
```

"""
--------数据处理--------
"""

```
x_train_reshape = x_train.reshape(-1,1)
x_test_reshape = x_test.reshape(-1,1) #改成二维 n*1

poly = PolynomialFeatures(degree=4)  #创建四次项式子
x_train_poly = poly.fit_transform(x_train_reshape)    #生成对应多项式矩阵
```

"""
--------训练模型和预测--------
"""

```
lr = LinearRegression()    #创建线性回归模型
lr.fit(x_train_poly,y_train)   #train 的预测训练
y_predict = lr.predict(x_train_poly)

x_test_poly = poly.fit_transform(x_test_reshape)  #test 的预测
y_predict_test = lr.predict(x_test_poly)
```

"""
--------结果可视化--------
"""

```
# 创建 1 行 2 列的子图
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

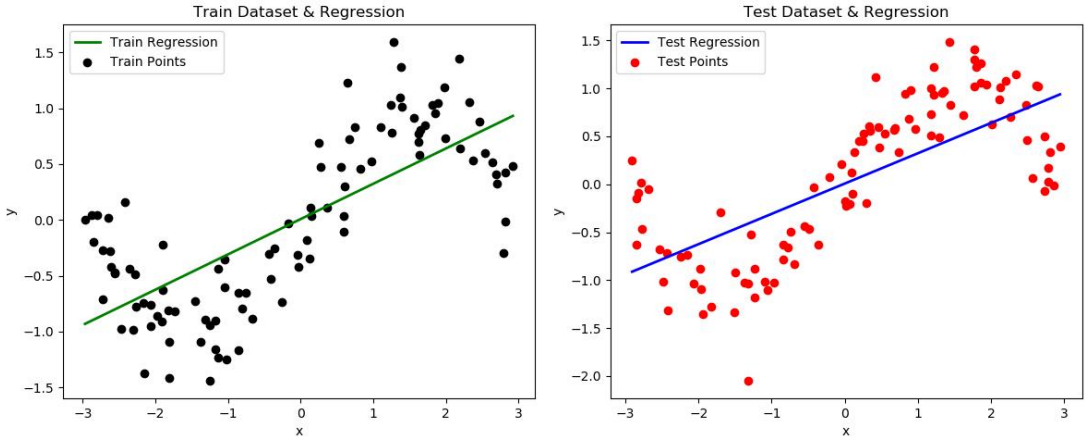
# 绘制训练集结果
axes[0].scatter(x_train, y_train, color='k', label='Train Points')
axes[0].plot(np.sort(x_train), y_predict_train[np.argsort(x_train)], linewidth=2, color='g', label='Train Regression')
axes[0].set_title("Train Dataset & Regression")
axes[0].set_xlabel("x")
```

```
9. axes[0].set_ylabel("y")
10.axes[0].legend(loc="best")
11.
12.# 绘制测试集结果
13.axes[1].scatter(x_test, y_test, color='r', label='Test Points')
14.axes[1].plot(np.sort(x_test), y_predict_test[np.argsort(x_test)],
    linewidth=2, color='b', label='Test Regression')
15.axes[1].set_title("Test Dataset & Regression")
16.axes[1].set_xlabel("x")
17.axes[1].set_ylabel("y")
18.axes[1].legend(loc="best")
19.
20.# 调整子图布局
21.plt.tight_layout()
22.plt.show()
```

六：实验结果

1. 欠拟合结果——图

当使用低阶多项式（如 degree=1）时，模型仅能学习数据的线性趋势，无法捕捉其中的非线性特征。此时，训练误差和测试误差都较大，模型的拟合能力较差，表现出典型的欠拟合现象。这表明模型过于简单，未能充分利用数据的信息。解决欠拟合的方法包括增加模型复杂度（如使用更高阶的多项式）、引入更多特征变量或调整超参数，以提升模型的表达能力。

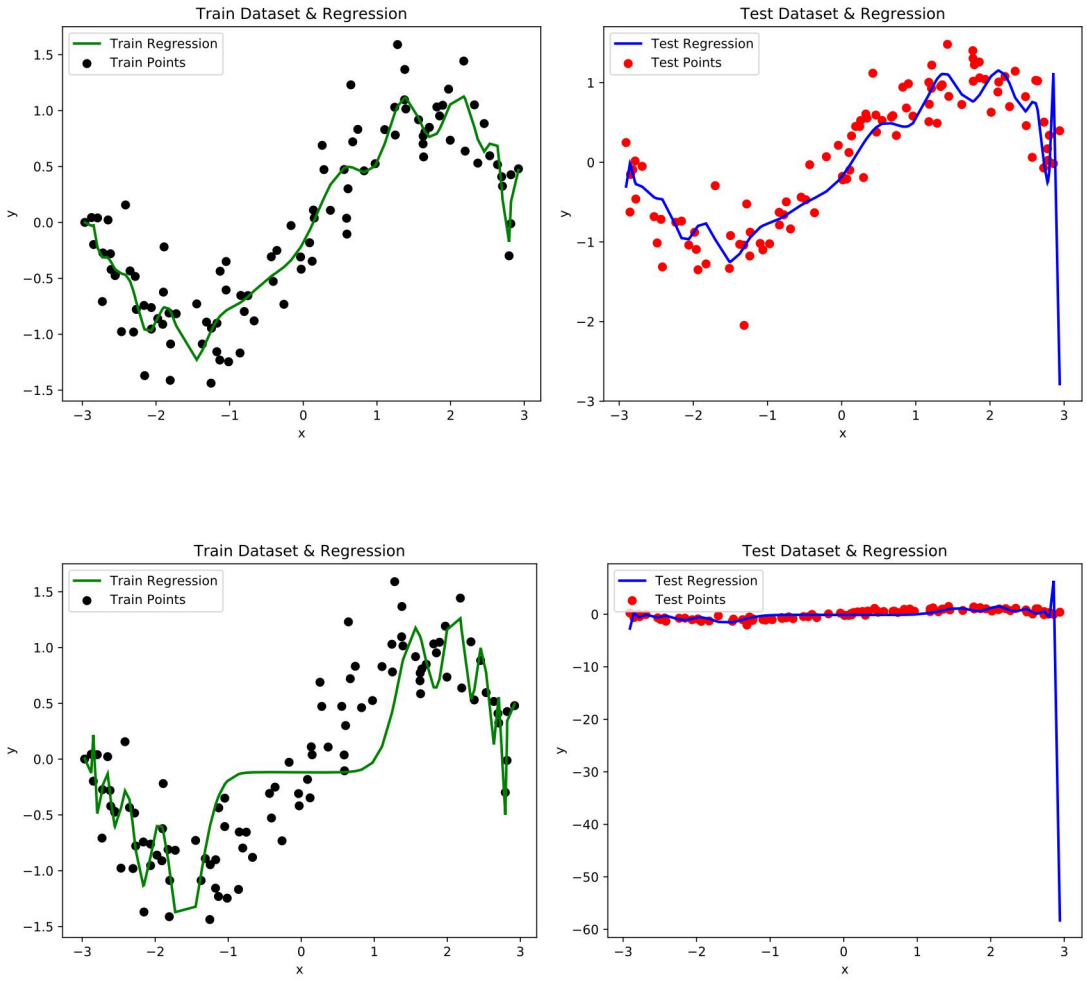


2. 过拟合结果——图

当多项式阶数过高（如 degree=40）时，模型可以高度拟合训练数据，甚至能够捕捉到数据中的噪声。这导致训练误差非常小，但测试误差较大，说明模型的泛化能力较弱。过拟合的本质是模型对训练数据学习过度，使得它在新数据上的表现变差。为缓解过拟合，可以采取以下措施：

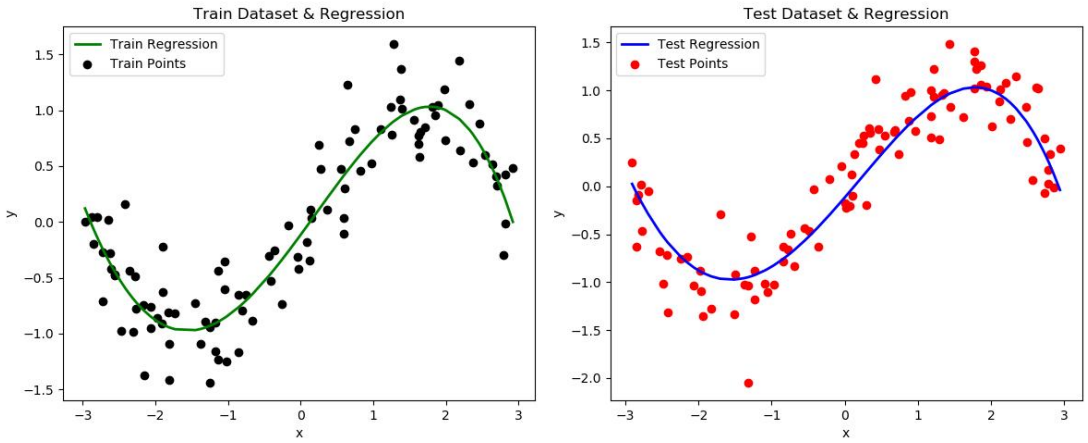
- 降低模型复杂度，如减少多项式阶数；
- 使用正则化方法（如 L1 正则化 Lasso 或 L2 正则化 Ridge）以限制参数的过大波动；
- 增加数据量，以减少模型对单个数据点的敏感性。

下图 1 所示在 degree=30 的时候，模型的测试拟合虽然在最后出现较大波动，但仍能吻合数据，而图二，几乎所有的测试点都在一条直线，而最终的末尾却指向了极小处,说明模型产生了过拟合。



3. 正好拟合结果——图

当选择适当阶数（如 degree=4 或 degree=5）时，模型既能较好地拟合数据的趋势，又能保持较低的训练误差和测试误差。此时，模型能够有效学习数据中的模式，并且不会过分依赖训练数据，具备较好的泛化能力。这说明合理选择模型复杂度可以在准确性和泛化能力之间取得平衡。



七：总结

1. 训练集、验证集和测试集的作用分别是什么？划分时的注意事项。

训练集用于模型的训练，通过调整模型的参数使其能够适应数据的内在规律；验证集用于在训练过程中优化超参数，防止模型的过拟合；测试集用于在训练完成后评估模型在未见过的数据上的表现，衡量其泛化能力。在数据划分时，应保证数据分布的一致性，避免信息泄漏。通常，数据按照 60%-80% 的比例划分为训练集，20%-40% 为验证集与测试集，且在时序数据中需要按照时间顺序划分。

2. 欠拟合和过拟合两种状态有什么区别？结合偏差与方差进行描述。

欠拟合是指模型过于简单，无法充分捕捉数据的复杂性，表现为高偏差和低方差，训练集和测试集的误差都较大。过拟合是指模型过度拟合训练数据，导致在训练集上的误差较小，但在测试集上误差较大，表现为低偏差和高方差。欠拟合通常因模型复杂度不足引起，而过拟合通常是由于模型过于复杂、训练数据不足或正则化不足造成的。

3. 模型的泛化能力是指什么？什么样的模型具有较好的泛化能力？

泛化能力指的是模型在未见过的数据上的表现能力，是评估模型性能的重要指标。具有较好泛化能力的模型能够有效地将学习到的规律应用于新样本。提高泛化能

力的方法包括增加训练数据量、采用数据增强、正则化（如 L1/L2 正则化）、交叉验证、控制模型复杂度（例如通过调整正则化参数）以及早停等技术。这些方法有助于防止过拟合，确保模型在新数据上的表现稳定。